Literatura rusa

Bridget Mendez Arano

May 7, 2026

1 Introduction

La aplicación "Literatura Rusa" fue creada en Flutter y Springboot. El tema "Literatura Rusa" fue elegido por el interés en obras de dostoyeski con la finalidad de irse extendiendo poco a poco en este genero literario. El objetivo fue pasar el proyecto de tweeter, adaptarlo a la aplicación, hacer funcional las reacciones y comentarios. Ademas permitir que se guarde a la base de datos, guardar el conteo de las reacciones y los usuarios que hicieron la publicacion. Como nube ocupamos claudinary y base de datos Aiven en PostgreSQL

2 Patrones de diseño ocupados

2.1 Patrones Arquitectonicos

2.1.1 MVC - Model View Controller

Se ocuparon carpetas como lo son repository, controllers y models. Se utilizaron para separar la estructura del sistema en capas. Los modelos representan los datos, los controladores reciben las peticiones HTTP y los repositorios se encargan del acceso a la base de datos.

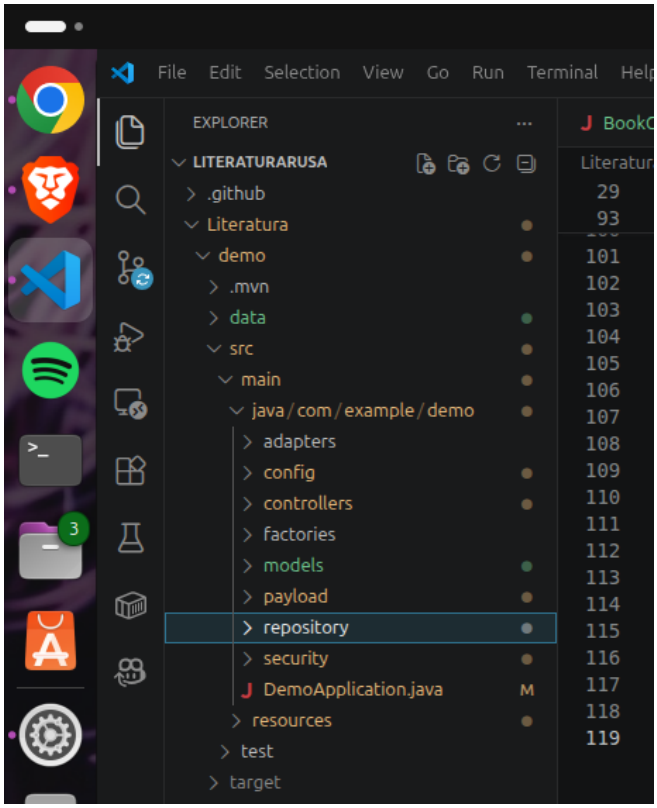

Figure 1: Estructura del backend en Spring Boot.

2.1.2 Capa de servicio

Esta carpeta se utilizo para agregar AuthService y BookService. AuthService se encarga del login, registro, token y cierre de sesión en cambio BookService se encarga de consultar libros, crear libros, comentar y reaccionar. Esto evita que las pantallas hagan directamente peticiones HTTP.

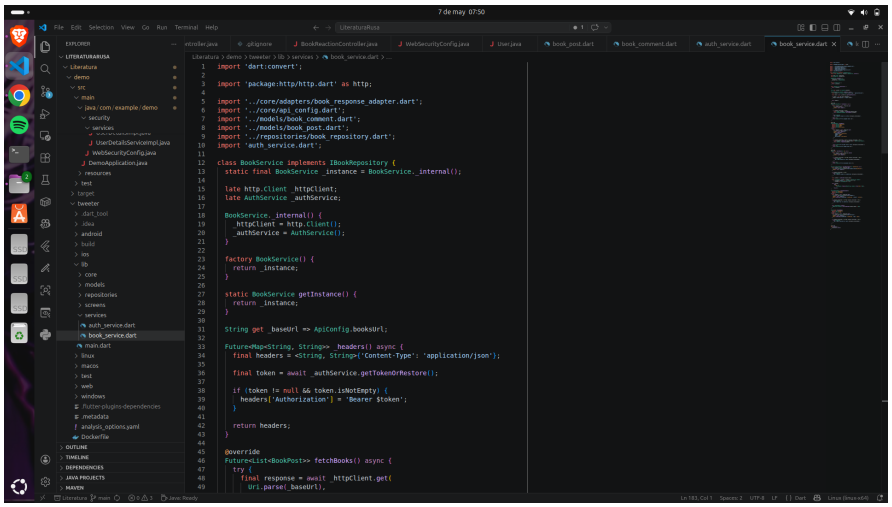

Figure 2: final books = await bookService.fetchBooks(); La pantalla solo pide los libros, pero no sabe todos los detalles de la conexión.

2.2 Patrones estructurales

2.2.1 FACADE

Este se ocupo principalmente en flutter en la carpeta facade en el archivo library-facade.dart, este se utilizó para simplificar la comunicación entre las pantallas y los servicios. Este en lugar de que HomeScreen o LoginScreen llamen directamente a AuthService o BookService, llaman a métodos de LibraryFacade. Este tiene una ventaja es que el sistema ya no necesita conocer todos los detalles internos de los servicios solo lo llama y ya.

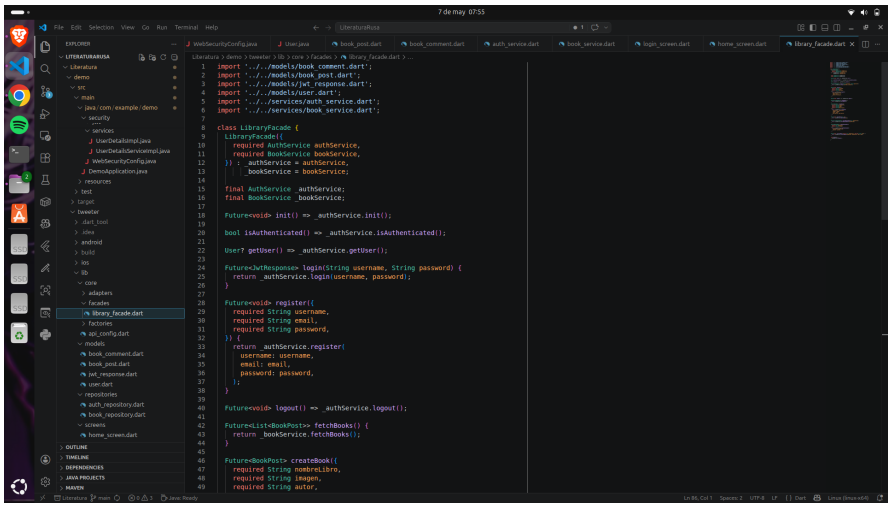

Figure 3: Todos esos métodos están concentrados en LibraryFacade, pero internamente la fachada llama a AuthService o BookService.

2.2.2 Adaptador o Adapter

Se utilizó para adaptar la respuesta del backend al formato que Flutter necesita. El backend puede devolver libros como lista directa o como respuesta paginada con content. El adaptador permite manejar ambos casos y convertirlos en objetos BookPost.

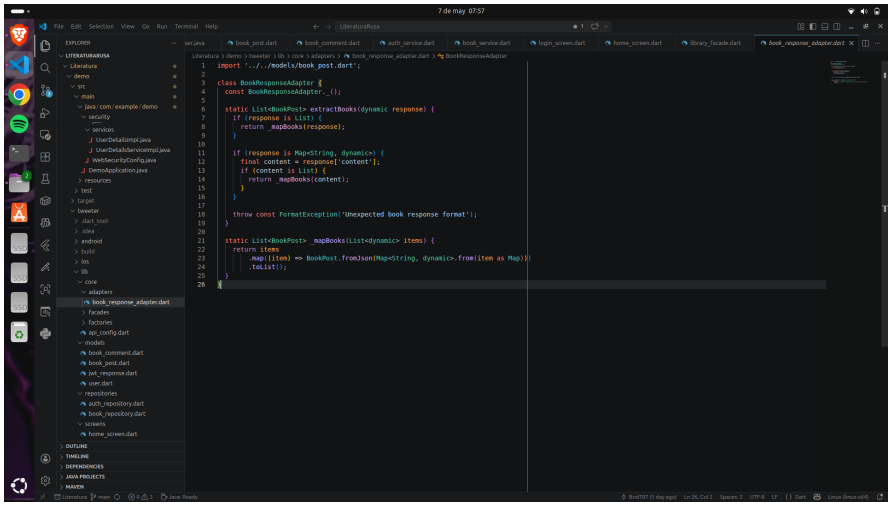

Figure 4: return BookResponseAdapter.extractBooks(jsonData); Con esto, el frontend puede trabajar siempre con una lista de libros, aunque el backend mande la información en formatos distintos.

2.2.3 DTO - Data Transfer Object

Se utilizó para enviar al frontend solo la información necesaria, sin exponer directamente las entidades de la base de datos.

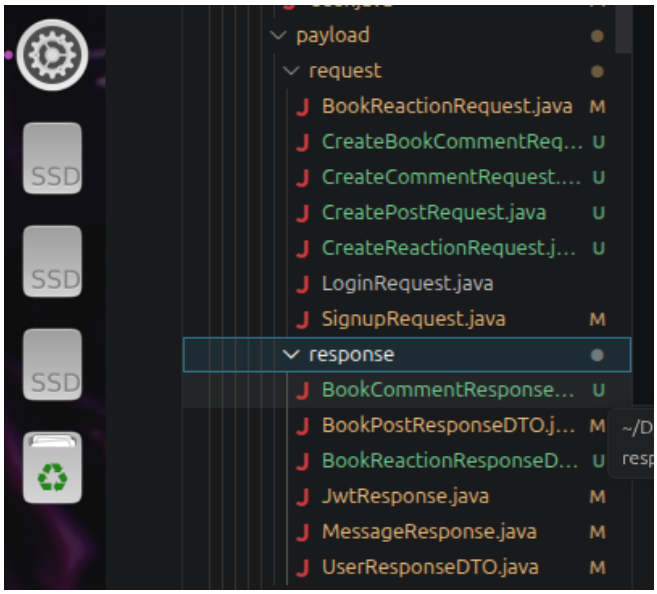

Figure 5: DTO

2.3 Patrones creacionales

Sirve para crear los objetos principales que necesita la aplicación, por ejemplo: AuthService, BookService y LibraryFacade.La idea es que en vez de crear esos servicios directamente en main.dart, se hace todo en una sola clase, es decir, main.dart no se preocupa por crear primero AuthService, luego BookService, y después pasarlos a LibraryFacade. Todo eso lo hace la fábrica.

2.4 Singleton

Se utilizó para mantener una única instancia de los servicios principales. Esto es importante especialmente en AuthService, porque ahí se guarda información de sesión como los tokens, la autentificacion de usuario

2.5 Patrones de acceso a datos

2.5.1 Repository

Se utilizó para separar el acceso a la base de datos de los controladores. Aqui BookPostRepository permite guardar y consultar libros, BookCommentRepository permite obtener comentarios asociados a un libro y BookReactionRepository permite guardar reacciones y contar cuántas tiene un libro.

2.5.2 Instancia única

El patrón Singleton se utilizó en Flutter en servicios como: AuthService, BookService. Este patrón permite que exista una sola instancia compartida de una clase.Este se usó porque AuthService maneja información importante de sesión, como el token JWT y el usuario autenticado. Si se crearan varias instancias diferentes de AuthService, podría haber inconsistencias, por ejemplo, que una instancia tenga token y otra no.

```
class AuthService implements IAuthRepository {
  static final AuthService _instance = AuthService._internal();

  late http.Client _httpClient;
  SharedPreferences? _prefs;

  String? _token;
  User? _user;

  AuthService._internal() {
    _httpClient = http.Client();
  }

  factory AuthService() {
    return _instance;
  }
```

Figure 6: Esto hace que cada vez que se llame a AuthService(), se devuelva la misma instancia.

2.6 SOLID

2.6.1 SRP — Single Responsibility Principle

Este principio dice que una clase debe tener una sola responsabilidad principal. En el proyecto se aplicó separando las clases por función.Por ejemplo:

AuthController se encarga del login y registro.
BookPostController se encarga de los libros.
BookCommentController se encarga de los comentarios de libros.
BookReactionController se encarga de las reacciones de libros.
En Flutter también se aplicó:
AuthService maneja autenticación.
BookService maneja libros, comentarios y reacciones.

2.6.2 OCP — Open/Closed Principle

Este principio dice que el sistema debe estar abierto para extenderse, pero cerrado para modificarse innecesariamente. En el proyecto se aplicó cuando se agregaron comentarios y reacciones a los libros. En lugar de meter toda esa lógica dentro de BookPostController, se crearon clases separadas:

BookComment, BookReaction, BookCommentController, BookReactionController, BookCommentRepository, BookReactionRepository

Esto permitió extender la funcionalidad del sistema sin sobrecargar la clase principal de libros.

2.6.3 LSP — Liskov Substitution Principle

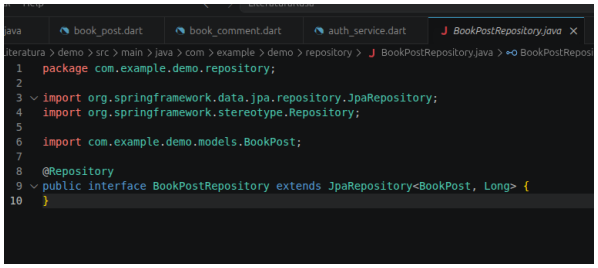

Figure 7: BookPostRepository extiende de JpaRepository, por lo tanto puede comportarse como un repositorio JPA válido dentro de Spring

2.6.4 ISP — Interface Segregation Principle

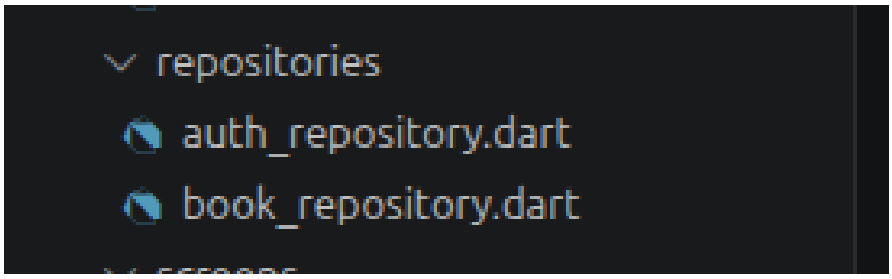

Figure 8: La idea es que la autenticación tenga su propio contrato y los libros tengan otro. Así, una clase relacionada con login no necesita depender de métodos para crear libros o comentar.

2.6.5 DIP — Dependency Inversion Principle

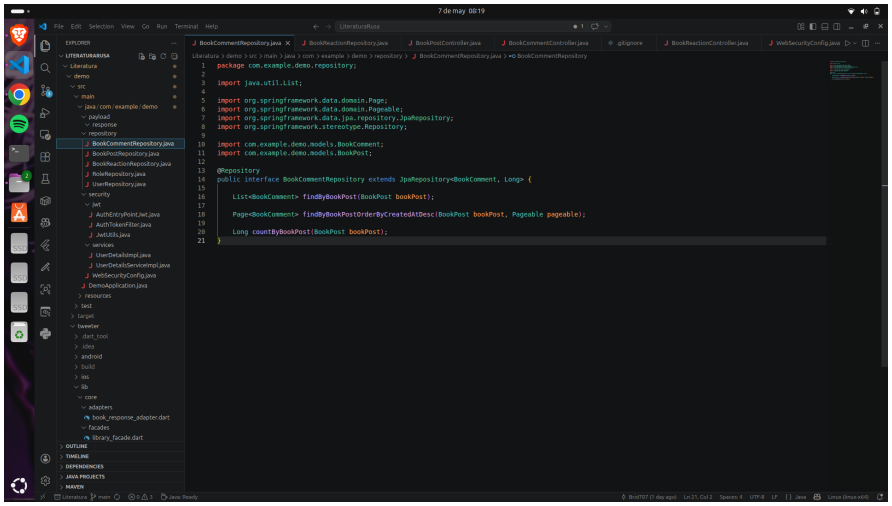

Figure 9: Esto significa que BookCommentController no crea manualmente los repositorios. Spring Boot se los entrega automáticamente.